

Thermal and Payload Aware Control of a Two Joint Robot Arm

Deep Q Learning for Variable Payload and Motor Temperature Constraints

ME5418 Final Project Proposal

Why

- Planar robot arms are used to position cameras, tools, and small parts in inspection and assembly systems. The same arm may carry tools or objects of different masses. A controller tuned for one mass can move too slowly, overshoot the target, or use unnecessary motor effort when the mass changes. Repeated motor effort also produces heat, so the arm must reach and hold its target without exhausting the available thermal capacity.
- This project studies a fixed base arm with two links, two rotary joints, and one motor at each joint. Both joints can be controlled directly. The task is end effector positioning and stable holding; it contains no obstacles, route selection, or path planning.

Existing and Conventional Methods

- A conventional controller measures the difference between the current and desired joint positions and computes motor commands from that error. This can work well after its settings have been tuned for a particular payload. A practical system can also store different settings for light and heavy tools, but it must first identify the payload correctly.
- Motor protection is commonly added as a separate rule that reduces or disables a motor after a temperature limit is reached. This rule prevents further heating but does not plan ahead. When the payload estimate is wrong, fixed controller settings and a reactive cutoff may produce slow motion, overshoot, or a shutdown during the final holding stage. The proposed agent will instead learn the longer term consequence of using either motor at each step.

Problem Statement

- The simulator will extend Gymnasium Reacher v5, a two link planar robot arm simulated in MuJoCo. The original environment already provides continuous robot dynamics, two independently actuated joints, a random target, and animation. We will retain these features while replacing the original continuous action input with a small set of discrete torque combinations.
- At the start of each episode, the target position and terminal payload are selected. Training uses three payload masses: empty, light, and heavy. A payload sensor reports one of these classes but occasionally gives an incorrect or uncertain reading. The robot must move its end effector into a circular target region and remain there for five consecutive steps.
- Each motor has its own temperature value. Applying torque raises the corresponding temperature; selecting zero torque allows it to cool. If one motor reaches the protection level, only that motor is temporarily disabled. This temperature value is a simple measure of accumulated motor stress, not a detailed physical heat transfer model. It keeps the environment understandable and avoids requiring motor thermodynamics.
- The project contribution is the task design rather than a new learning algorithm. Standard Reacher mainly rewards target distance and low control effort. Our environment combines variable payload, imperfect payload sensing, two independent motor temperature limits, individual motor shutdown, discrete two joint torque commands, and a five step holding requirement in one control task.

Reinforcement Learning Setup

- **State space:**

- Robot motion: the angle of each joint is divided into eight ranges and each angular velocity into five ranges. These values describe the arm configuration and the direction and speed of motion.
- Target information: the direction from the end effector to the target is divided into eight sectors and the distance into far, medium, and near. These values tell the agent where corrective movement is required without giving it a precomputed solution.
- Task constraints: each motor temperature has five levels; the payload reading is empty, light, heavy, or uncertain; the hold counter ranges from zero to five; and remaining time has five levels. These values tell the agent whether it should keep moving, reduce effort, cool a motor, or complete the hold more urgently.
- Before entering the neural network, each category is converted into a short sequence of zeros with a single one marking the current category. This keeps the observation fully discrete while allowing a small network to process it.

- **Action space:** each joint can receive negative, zero, or positive torque. Combining the two joints gives nine discrete actions. The actions remain low level: the agent must discover how to coordinate both joints, brake near the target, hold position, and rest an overheated motor.

- **Reward structure:** successful completion of the five step hold receives the largest reward. Moving closer to the target and completing each hold step provide smaller guiding rewards. Motor use receives a small cost, while leaving the target during holding, overheating, protection shutdown, and timeout receive larger penalties. This order keeps task completion as the main objective and prevents the agent from earning a high return merely by moving near the target or remaining idle.

- **Neural network:** a small network with fully connected layers receives the encoded state and produces nine estimates, one for each action. Each estimate represents the predicted total future reward after selecting that action. The agent normally selects the action with the highest estimate.

Learning Algorithm

- We will implement Deep Q Learning, abbreviated as DQN, using the PyTorch neural network software library. DQN is suitable because the action space contains nine discrete choices and a torque command can have both an immediate mechanical effect and a delayed temperature cost.
- During training, the agent will sometimes try a random action to discover new behavior and will otherwise select the action with the highest predicted value. Each interaction records the current state, action, reward, next state, and whether the episode has ended. Training samples will be drawn from this memory so the network learns from varied past situations instead of only the latest step.
- A second copy of the network will generate stable training targets and will be updated less often than the main network. We will write the action selection, interaction memory, target calculation, network update, saving, loading, and evaluation code ourselves instead of using a prebuilt reinforcement learning package.

Experiments and Envisioned Results

- We will compare four methods: random actions; a manually written controller that moves toward the target and stops a hot motor; a DQN that does not observe payload or temperature; and the complete DQN. These comparisons test whether learning improves on simple rules and whether payload and temperature information produces a measurable benefit.
- Tests will include a fixed light payload, randomized training payloads, stricter temperature limits, and payload sensor error rates of zero, ten, and twenty percent. Generalization will be tested with an intermediate payload mass that is not used during training; its sensor value will be reported as uncertain so the observation remains consistent with the state definition.
- The two primary measures are task success rate and the percentage of episodes that trigger motor protection. Secondary measures are completion time, number of nonzero torque commands, failed hold attempts, and success rate for each payload. Every method will be tested with several random seeds, and the mean and variation will be reported.
- We expect the complete DQN to coordinate the two motors differently for light and heavy payloads and to use zero torque before a protection event when cooling is worthwhile. This is a hypothesis, not an assumed result. If the manual controller remains more reliable under sensor error, the experiments will identify the conditions in which the learned policy loses its advantage.

- **References**

- [1] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529-533. <https://doi.org/10.1038/nature14236>
- [2] Spong, M. W. (1995). The swing up control problem for the Acrobot. *IEEE Control Systems Magazine*, 15(1), 49-55. <https://doi.org/10.1109/37.341864>
- [3] Kober, J., Bagnell, J. A., and Peters, J. (2013). Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11), 1238-1274. <https://doi.org/10.1177/0278364913495721>